

RCNN Object Detection

Bevezető

A notebook egy RCNN algoritmust valósítja meg, objektumok detektálására képeken. Az alapötlet: régiókat generálunk a képen, minden régióból CNN feature-t nyerünk ki, majd SVM-mel osztályozunk.

0. Környezet beállítása

GPU ellenőrzés:

- 1 Ha nincs GPU, azonnal tudod, hogy a tanítás lassú lesz, még mielőtt a dataset letöltés és a feature kinyerés elindulna.
- 2 Az eszköz változó. Visszajelez, hogy milyen eszközt használ.
- 3 Colab alapértelmezésben CPU runtime-on indul, a GPU-t manuálisan kell bekapcsolni.

Kulcs könyvtárak:

Könyvtár:	Mire használjuk:
torch + torchvision	ResNet50/ResNet18 backbone, GPU támogatás
cv2 (opencv-contrib-python)	Selective Search régiógenerálás
sklearn	SVM osztályozó és Ridge regresszor
kagglehub	Pascal VOC 2012 adathalmaz letöltése
PIL	Képbetöltés, régiók kivágása
matplotlib	Vizualizáció: bbox-ok, hisztogramok, demó

1. Adathalmaz feltérképezése

- a) Adathalmaz letöltése és szerkezete

A Pascal VOC 2012 adathalmaz struktúrája:

Annotations/	XML fájlok: minden képhez tartozó objektumok listája
JPEGImages/	Képek (.jpg)
ImageSets/Main/	Train/val split fájlok osztályonként

A Pascal VOC ImageSets/Main/ mappában osztályonként külön fájlok vannak: aeroplane_train.txt, aeroplane_val.txt, bicycle_train.txt, stb. (összesen 20 osztály $\times$ 2 = 40 fájl).

A kód beolvassa az összes train.txt fájlt, összegyűjti a kép-azonosítókat, majd set() -tel egyedi értékekre redukálja. Ugyanezt megcsinálja a val.txt fájlokkal is. *(Egy kép több osztályt is tartalmazhat (pl. a 2008_000123.jpg-n lehet person és bicycle is), ezért ugyanaz a képezonosító több osztály train fájljában is szerepelhet. A set() ezeket a duplikátumokat szűri ki.)*

b) XML annotáció elemzése

Minden képhez egy XML fájl tartozik, ami így néz ki (leegyszerűsítve):

```
<annotation>
  <size><width>500</width><height>375</height></size>
  <object>
    <name>person</name>
    <bndbox><xmin>10</xmin><ymin>20</ymin><xmax>200</xmax><y-
      ax>350</ymax></bndbox>
  </object>
</annotation>
```

A parse_voc_xml() ezt alakítja át. Visszaadja a két sarokpont koordinátáit és a kép méretét: [('person', [10, 20, 200, 350]), ...] formátumban.

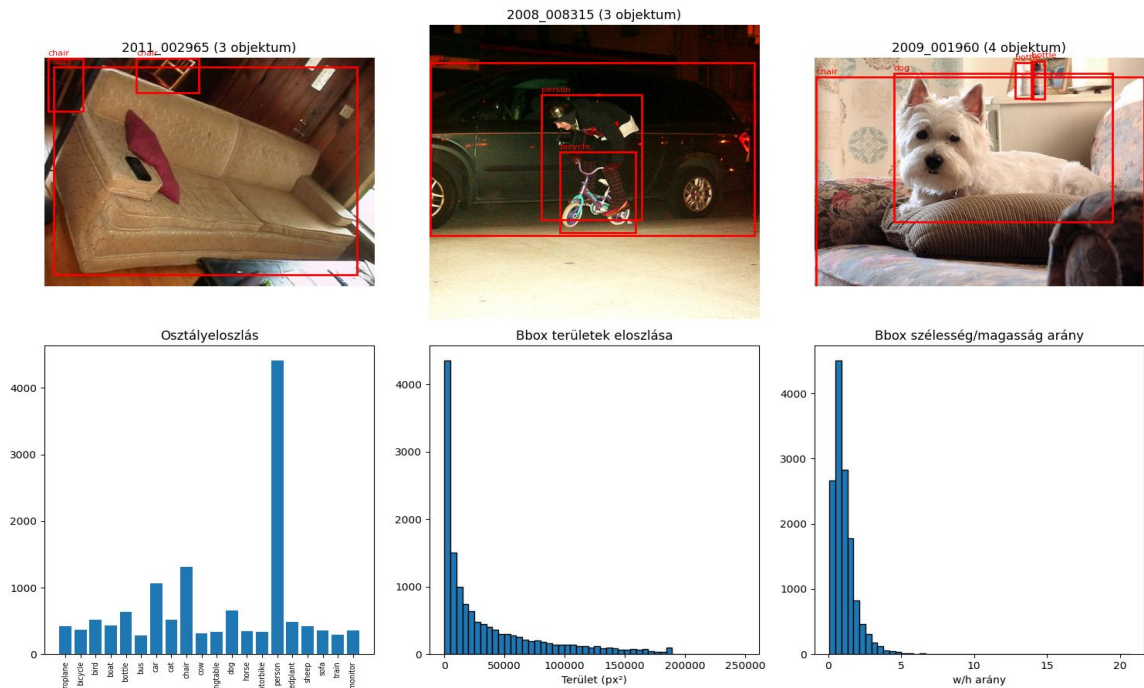
c) Statisztikák és mintavizualizáció

ciklus végigmegy 5000 train képen és kiszámolja:

- Osztályeloszlás: melyik osztályból hány példány van
- Bbox területek: $(x_{\max} - x_{\min}) * (y_{\max} - y_{\min})$
- Bbox arányok: szélesség / magasság
- Képméreteket: min/max méret

Ezután mindezt vizualizálja:

- 3 minta kép a bbox-okkal
- Kiválasztott 5000 adathalmaz hisztogramja: Osztályeloszlás, Bbox terület eloszlás, Bbox arány szerint



Minta képek statisztikái

2. RCNN háló létrehozása

Az RCNN három fő komponensből áll.

a) FeatureExtractor (ResNet50 backbone):

ResNet50 hálózathoz a osztályozó rétegét levágtuk. Elég nekünk a jellemzőkinyerő része, a megkapott jellemző osztályozását mi vegezzük.

Feature map: Amikor egy kép bekerül a ResNet50 backbone-jába, a konvolúciós rétegek elkezdik feldolgozni. Ahogy a kép halad előre a rétegeken, a felbontása (szélesség és magasság) egyre kisebb lesz, viszont egyre több "csatornát" kap, amelyek egy-egy felismert jellemzőt (élek, formák, textúrák) képviselnek. Ezeket a többdimenziós rácsokat hívjuk *feature map*-eknek.

Avgpool: A hálózatban a konvolúciós rétegek után következik avgpool réteg. Ennek az a feladata, hogy a feature map-et (a $7 \times 7 \times 2048$ -as rácsot) "kilapítsa" egy egyszerű, egydimenziós listává. Ezt úgy csinálja, hogy a 49db térbeli értéket átlagolja. Az eredmény egy 2048 elemű vektor lesz.

Végül az osztályozó réteg levágás után egy 2048 dimenziós feature vektort kapunk minden képrégióból. Ez egy tömör, szemantikus leírása a régió tartalmának. (az *ImageNet* előtanítás miatt már "érti" a vizuális fogalmakat.)

b) Képtranzformáció és régió feature kinyerése:

Kép-előfeldolgozása:

1. átméretezi a bejövő képet pontosan 224×224 pixelre ResNet-hez;
2. PyTorch-al a képet átalakítjuk matematikai adatszerkezeté;

3. Statisztikai normalizáció, amely a színek (Piros, Zöld, Kék csatornák) eloszlását állítja be.

Régiók kivágása, átméretezése és feature kinyerése batch-elve:

1. Régiókat vág ki az eredeti képből: `image.crop((x, y, x+w, y+h))`
2. Kivágott régiókat átranszformálja
3. Batch-be rendezi (64 régiót egyszerre)
4. Végigfuttatja a ResNet50-en
5. Összegyűjti a 2048 dimenziós feature vektorokat

Kimenete: numpy tömb ($N_{\text{régió}} \times 2048$)

c) Az RCNN modell:

1. Jellemzők kinyerése
2. Osztályozás: Megpróbálja eldönteni, hogy az adott régióban milyen objektum van SGDClassifier algoritmussal.
3. Doboz finomítás: Mivel az eredeti javasolt régiók általában pontatlanok (nem simulnak rá tökéletesen a tárgyra), a kód kiszámolja, hogyan kell "eltolni" és "átméretezni" a dobozt, hogy az tökéletesen illeszkedjen a képre.
4. Predikció új képen: Feature kinyerés minden régióra. Ezután 20 darab betanított SVM pontozza a régiókat. Utána a Ridge regresszorok megjósolják pontosítják a dobozt.

3. Adatelőkészítés

a) IoU (Intersection over Union) és Selective Search

Az IoU megmutatja, hogy a gép által rajzolt határoló doboz (a *jósolt bbox*) mennyire fedí át a valóságos, ember által megadott dobozt.

Selective Search pedig a javasolt dobozokat (Region Proposals) megkeresi a képen, mielőtt a hálózat megvizsgálja őket.

Először túlszegmentálja a képet, aztán hasonló szomszédos régiókat szín, textúra, méret alapján összevonja és minden összevonásnál az új régiót elmenti, mint régió javaslatot.

b) Tanítóadatok generálása

Minden képre az alábbi műveleteket hajtja végre:

1. Betöltjük a képet és az XML annotációt
2. Selective Search $\rightarrow$ régió javaslatok
3. Minden régióból ResNet50 feature kinyerés
4. Minden régióra megkeressük a legjobb IoU-jú objektumot
5. Címkézés:
 - $\text{IoU} \geq \text{iou_pos} (0.5)$: POZITÍV minta: `feature + osztály_címke + bbox`

- $\text{IoU} < \text{iou_neg} (0.3)$: NEGATÍV minta (háttér): feature + címke=-1
- $0.3 \leq \text{IoU} < 0.5$: KIDOBJUK: A kettős küszöb (0.5 és 0.3) lényege: a bizonytalan régiókat (amik félig fednek át egy objektummal) nem használjuk, mert összezavarnák az osztályozót.

Egy előre lefoglalt memmap fájlba (feature_cache/features.dat) írjuk a feature vektorokat float32 formátumban. Ezután negatív alulmintavételezéssel *(ha a negatív minták száma > 3x pozitív, véletlenszerűen eldobjuk a felesleget)* optimalizáljuk az adatok mennyiségét.

4. Tanítás és kiértékelés

a) Tanítás:

5000 train kép, képenként max 200 régió → kb.1000000 régió feldolgozása.

Tanítási folyamat:

1. prepare_training_data() → train_features (N x 2048), train_labels (N)
↓
2. rcnn.fit_svm(train_features, train_labels) → Mind a 20 osztályra:
StandardScaler + SGDClassifier
↓
3. rcnn.fit_regressors(pozitív_minták) → Minden osztályra (ahol ≥ 10 minta):
Ridge regresszió

b) Kiértékelés — mAP számítás

1. lépés - Detekciók gyűjtése:

Minden validációs képre:

- 500 régió generálása Selective Search-csel
- Minden régióra predikció (osztály + score)
- Top-20 detekció megtartása osztályonként

2. lépés - Precision-Recall görbe osztályonként:

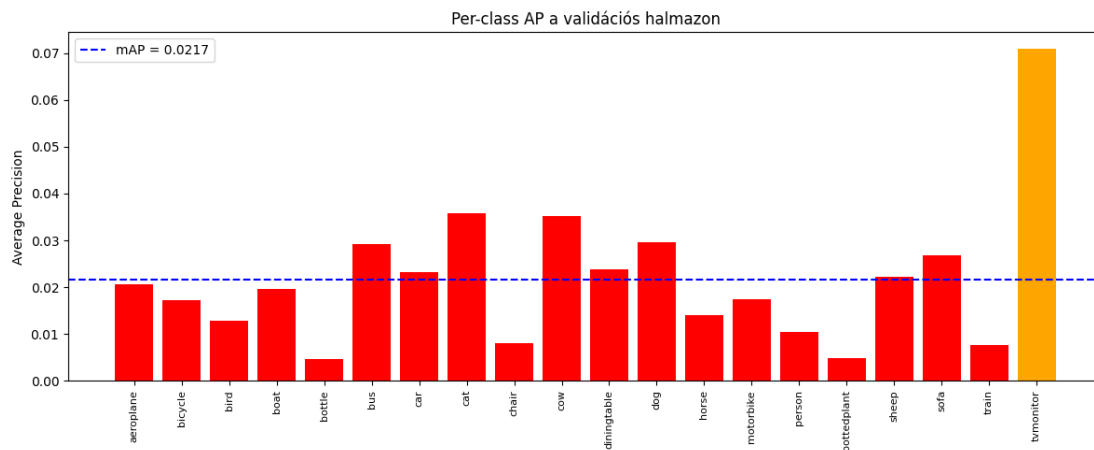
Detekciók rendezése score szerint csökkenőbe. Minden detekcióra az alábbi értékelést használjuk: ha a legjobb $\text{IoU} \geq 0.5$ valamelyik még nem használt GT-vel, akkor True Positive, más esetekben False positive.

3. lépés – All point Average Precision

4. lépés – mAP kiszámítása

c) Eredmények vizualizációja

AP érték vizualizálása: zöld > 0.1, narancs > 0.05, piros: gyenge.



5000 train képpel 1000 val képen

d) Eredmény értékelése

A validációs halmazon mért mAP érték 0.0217 ($\text{IoU} \geq 0.5$ mellett). Bár ez az érték jelentősen elmarad az eredeti RCNN-cikk eredményétől (Girshick et al., 2014: ~ 0.31 VOC 2010-en), a kísérlet több, az architektúra korlátaira és a megvalósítási tradeoff-okra vonatkozó tanulságot szolgáltatott.

A modell korlátainak elemzése:

1. A backbone fine-tuning hiánya: Az ImageNet-en előtanított ResNet50 az ImageNet ezerosztályos klasszifikációs feladatára van optimalizálva, nem pedig 20 osztályos detekcióra. Az eredeti RCNN-cikk szerint a backbone fine-tuningja 8 mAP százalékpont nagyságrendű növekedést okoz, így ennek elhagyása valószínűleg az eredmények legfőbb korlátozó tényezője.
2. Az SVM-alapú osztályozó korlátai. A lineáris SVM csak lineárisan szeparálható feladatokra hatékony a feature térben. Magas intra-class variance esetén (pl. person, horse) ez az architektúra elvi szinten alulmúlja a modernebb, end-to-end tanított softmax fejeket.
3. A Selective Search recall plafonja. A Selective Search recallja Pascal VOC-on Fast módban $\sim 90\%$ környékére tehető. Ez azt jelenti, hogy a célobjektumok $\sim 10\%$ -ára egyetlen elég jó proposal sem készül, vagyis az AP felső plafonja már magán a proposal mechanizmuson múlik, függetlenül a downstream osztályozó minőségétől.
4. Korlátozott adatmennyiség. Az 5000 tanító képből származó $\sim 30\,000$ minta húsz osztály között elosztva osztályonként átlagosan ~ 1500 pozitív példát eredményez (bizonyos osztályoknál sokkal kevesebbet). 2048-dimenziós feature térben ez kevés egy jól általánosító SVM-modellhez.

5. Hiperparaméter optimalizálás és architektúra módosítás

a) IoU küszöb hangolás

Három különböző IoU pozitív küszöbvel tanít:

IoU	mAP	Magyarázat
0.4	0.0696	Több pozitív minta, de zajosabb
0.5	0.0639	Eredeti RCNN beállítás
0.6	0.0372	Kevesebb, de tisztább pozitív

```
Optimalizálási összefoglaló
IoU=0.4: mAP=0.0696
IoU=0.5: mAP=0.0639
IoU=0.6: mAP=0.0372

Legjobb: IoU=0.4 (mAP=0.0696)
```

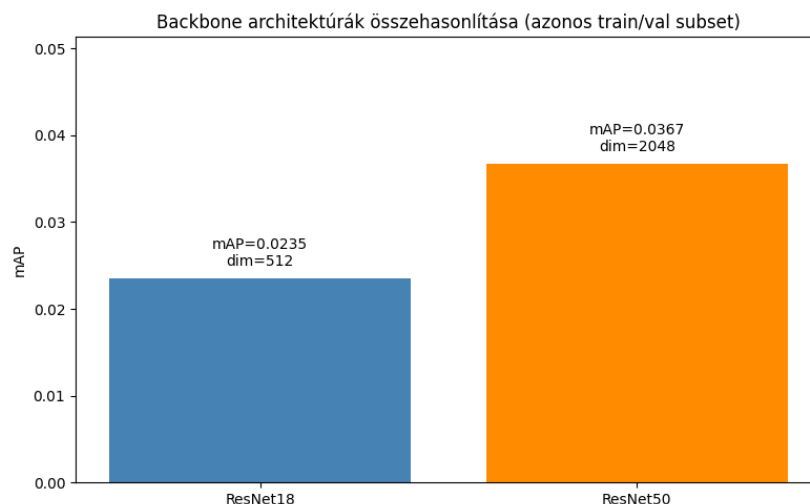
500 train + 100 val képen összehasonlítva.

b) ResNet18 vs ResNet50

Két backbone architektúra összehasonlítása:

Tulajdonság	ResNet18	ResNet50
Feature dimenzió	512	2048
Rétegek száma	18	50
mAP	0.0235	0.0367
Relatív különbség	—	+56.2%

Mindkettő ImageNet előtanított.



ResNet50-on alapuló RCNN modell mAP-ja 56%-kal magasabb (0.0367 vs 0.0235), mint a ResNet18-on alapuló modellé. Ez igazolja, hogy a backbone mélysége és

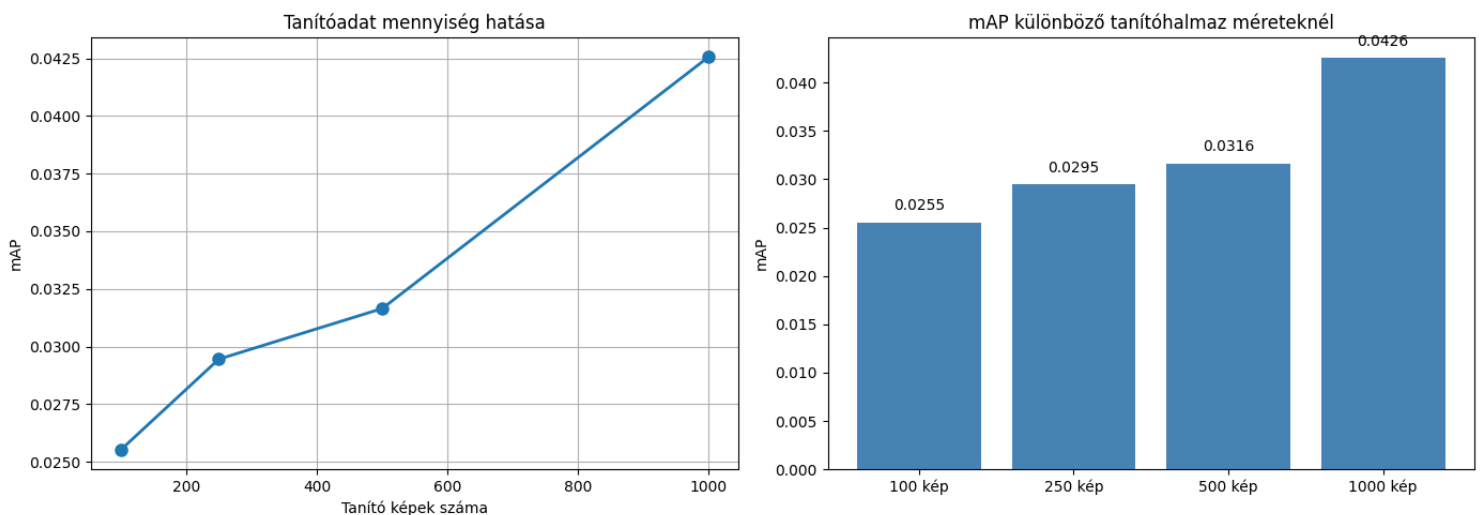
feature dimenziója jelentősen befolyásolja a detektálási teljesítményt, és összhangban van a klasszikus deep learning architektúrák ImageNet-en mért eredményeivel.

6. Tanítóadat mennyiség vizsgálata

a) Adatmennyiség hatása

Négy különböző tanítóhalmaz méret:

Tanító képek	mAP	Relatív
100	0.0255	10%
250	0.0295	25%
500	0.0316	50%
1000	0.0426	100%



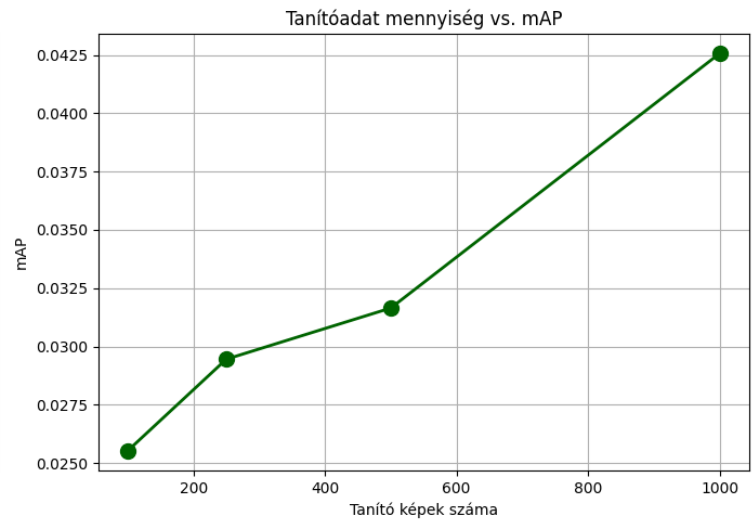
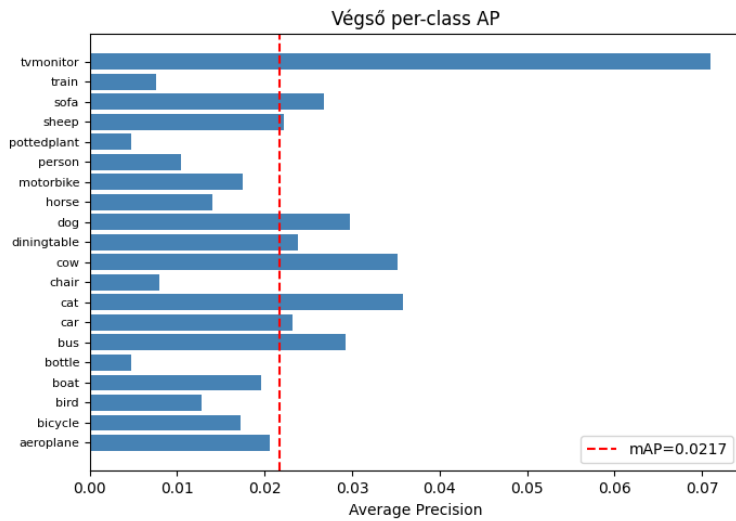
Ez a jellegzetes telítődési görbe nem a RCNN specifikuma, hanem általános ML jelenség: lineáris osztályozóknál a mintaszám logaritmikus növelésével érhető el lineáris pontosságjavulás.

b) Osztályszám variáció

Azt vizsgálja, hogy kevesebb osztály esetén hogyan alakul a minták száma osztályonként. 5, 10, majd 20 osztállyal tanít.

Konklúzió: több osztály esetén nagyobb tanítóhalmaz kell.

7. Végeredmény dokumentálása



Következtetések:

1. A RCNN működőképes objektumdetektálásra, de lassú (Selective Search + külön SVM tanítás).
2. A ResNet50 backbone jó alapot ad, a feature-ök diszkriminatívak.
3. A tanítóadatok növelése javítja a teljesítményt, de csökkenő hozadékkal.
4. Több osztály esetén több tanítóadat szükséges.
5. A modell leginkább kis objektumok és ritka osztályok detektálása alkalmas.